Sanchalak

A Cross-Source Knowledge Agent for FlytBase Solutions Engineering

Technical Documentation & Build Report

August 15, 2026

Problem Statement: Solutions Engineer Track — PS2: Knowledge Base Over Customer Data
Built for: FlytBase Hackathon
Integration: FlytBase live product documentation & release notes (docs.flytbase.com, releases.flytbase.com)

Abstract

Sanchalak (Sanskrit: *the one who conducts / operates*) is a conversational knowledge agent built to answer questions that span two otherwise disconnected worlds: a company's internal customer data (accounts, issues, feature requests, tasks, meeting notes) and its public product documentation. Rather than requiring a Solutions Engineer to manually cross-reference a CRM-like corpus against docs and release notes, Sanchalak retrieves from both sources live, combines them when a question requires it, and grounds every claim it makes in a specific record or documentation page — refusing to answer rather than guessing when it cannot find sufficient grounding. This document describes the problem, the system architecture, the AI/ML techniques used, the full technology stack, and the reasoning behind each major design decision.

Contents

1 Problem Statement

1.1 The Challenge

Customer data (accounts, issues, feature requests, tasks, meeting notes) and product knowledge (documentation, release notes) live in separate systems. There is no single place to ask a question that spans both — answering such a question today means manually checking multiple systems and combining what is found by hand.

1.2 Why It Matters

A question that touches both customer history and product documentation currently has to be answered fresh every time. This manual stitching does not get faster as the underlying data grows, and the same ground is covered repeatedly by different people instead of becoming easier to search over time.

1.3 Success Criteria (as specified by the problem statement)

- Answer questions using the provided customer-data corpus.
- Answer questions using *live* product documentation and release notes — not a static, pre-scraped copy.
- Combine both sources in a single answer when required, e.g. *"do accounts that requested a feature already have platform support for it, per the docs?"*
- Ground every answer in a specific record or documentation page; state clearly when there is insufficient information rather than guessing.
- Reflect an updated corpus (added, changed, or removed records) without requiring a full manual rebuild.

2 System Overview

Sanchalak is architected as a three-stage retrieval-and-synthesis pipeline rather than an open-ended autonomous agent loop. This was a deliberate design decision: the task — classify a question's source needs, retrieve, synthesize with citations — is bounded and small, and does not benefit from the complexity or unpredictability of a general agentic harness.

FlytBase Integration

Sanchalak integrates directly and *live* with FlytBase's public product surfaces:

- **docs.flytbase.com** — fetched at query time for product documentation grounding.
- **releases.flytbase.com** — fetched at query time for release-note grounding, enabling the contradiction-detection feature (Section 5.4).

No static copy of either site is stored; every docs-grounded answer reflects the live state of FlytBase's public documentation at the moment the question is asked. Per hackathon scope rules, Sanchalak does *not* connect to FlytBase's actual internal production systems — the customer-data side of the system operates exclusively on the provided synthetic corpus.

2.1 Pipeline Stages

1. **Tier 1 — Deterministic Retrieval (no LLM).** Simple, filterable questions (e.g. "What are the open issues for *Meridian AgriTech*?") are answered by direct structured lookup against tagged metadata, with zero model inference required. This removes the LLM as a single point of failure for the most common query shape and is instantaneous.

2. **Tier 2 — LLM Synthesis (primary).** Questions requiring genuine synthesis, cross-source combination, or natural-language reasoning are routed to the primary LLM, which is given tool-use access to two retrieval functions and must produce a cited, grounded answer.

3. **Tier 3 — LLM Fallback.** If the primary model fails, times out, or produces unreliable tool-calls, the request is retried against a fallback LLM with an identical tool-calling contract, so no provider-specific branching logic is needed downstream.

3 Architecture

3.1 Ingestion Pipeline (Customer Corpus)

The provided synthetic customer-data folder (accounts, issues, feature requests, tasks, meeting notes) is parsed into typed records, chunked, embedded, and stored in a local vector store with structured metadata (record type, account name, industry, plan tier, date, category). A delta-reindexing function re-embeds only changed or newly added records and removes deleted ones, avoiding a full corpus rebuild on every update — this directly satisfies the "stays current without a full rebuild" requirement and is demonstrated live by editing a record and re-querying.

3.2 Live Documentation Retrieval

A dedicated retrieval tool queries docs.flytbase.com and releases.flytbase.com at query time, returning passages tagged with their source URL. A short-lived cache provides a graceful fallback if a live fetch fails, with a visible "stale/cached" indicator surfaced to the user rather than a raw error.

3.3 Query Router / Orchestrator

Given an incoming question, the orchestrator classifies whether it requires the customer corpus, the live docs, or both, then invokes the corresponding retrieval tool(s) before handing results to the synthesis stage.

3.4 Grounded Synthesis & Contradiction Detection

The synthesis stage enforces a strict *cite-or-refuse* policy: every factual claim must carry a citation to a specific record ID or documentation URL, or the system must explicitly state that it lacks sufficient information. A secondary check cross-references customer-data feature requests against release notes to flag contradictions — e.g. a feature marked "requested" in customer data that is already listed as shipped in the release notes.

3.5 Frontend

A chat interface renders agent answers with inline, source-colored **citation chips**, each connected via a colored line to a source rail listing the underlying records/pages used — a direct visual rendering of the system's grounding guarantee. Four distinct UI states are implemented:

normal grounded answer, refusal (deliberately calm, non-error styling), stale/cached fallback indicator, and contradiction banner (reserved exclusively for genuine contradictions, to preserve its visual significance).

4 Technology Stack

Layer	Technology	Role
Frontend	React	Chat UI, citation-chip rendering, source rail, streaming answer display
Backend	Python + FastAPI	Query routing, retrieval orchestration, synthesis endpoint, delta-reindex API
Vector Store	ChromaDB	Local embedded vector storage for the customer-data corpus, with metadata filtering
Primary LLM	Gemini 3.6 Flash	Query classification, tool-orchestrated retrieval, grounded answer synthesis, contradiction detection
Fallback LLM	Grok	Automatic fallback when the primary model fails, times out, or produces unreliable tool-calls
Frontend Deployment	Vercel	Hosting for the React client
Backend Deployment	Persistent-process host (Railway/Render/Fly.io class)	Hosting for FastAPI + ChromaDB, required for persistent local vector storage that Vercel's serverless model cannot provide
Live Data Sources	docs.flytbase.com, releases.flytbase.com	Fetched live at query time for documentation and release-note grounding

4.1 Development Tooling

Tool	Model Used	Purpose in the Build
Antigravity	Claude Opus	Primary coding agent — initial implementation of the ingestion pipeline, retrieval tools, orchestrator, and frontend components
OpenCode	DeepSeek V4	Refinement pass — bug fixing, debugging retrieval/routing issues, and repository comprehension when picking the project back up mid-build

5 AI / ML Concepts & Terminology

This section defines the core AI/ML techniques used in Sanchalak, for reviewers less familiar with retrieval-augmented systems.

Retrieval-Augmented Generation (RAG) — The overall pattern Sanchalak implements: rather than relying on an LLM's internal training knowledge, relevant facts are retrieved from

external sources (the customer corpus, live docs) at query time and supplied to the model as context, which it must ground its answer in.

Embeddings — Numerical vector representations of text that capture semantic meaning, allowing "similar meaning" text to be found even when exact keywords differ. Used to index and search the customer-data corpus.

Vector Store / Vector Database — A database optimized for storing embeddings and performing similarity search (ChromaDB, in this build), typically combined with structured metadata filters (e.g. industry, date, record type).

Chunking — Splitting longer records (e.g. meeting notes) into smaller passages before embedding, so retrieval returns focused, relevant excerpts rather than entire documents.

Tool Use / Function Calling — An LLM capability allowing the model to invoke defined functions (in this build: `search_customer_data(query)` and `fetch_docs(query)`) rather than generating free-form text alone, enabling the orchestrator pattern described in Section 3.

Grounding — The requirement that every factual claim in a model's answer be traceable to a specific, cited source. Sanchalak enforces this as a hard constraint rather than a best-effort behavior.

Hallucination — The failure mode RAG and grounding are designed to prevent: an LLM generating plausible-sounding but unsupported or fabricated claims. Sanchalak's cite-or-refuse policy is the primary mitigation.

Cite-or-Refuse Policy — A synthesis-time constraint requiring the model to either attach a citation to a claim or explicitly state it lacks sufficient information — never to assert an ungrounded claim.

Delta Reindexing / Incremental Indexing — Re-embedding only records that have changed, been added, or been removed, rather than rebuilding the entire vector index on every update — critical for the "stays current without full rebuild" requirement.

Query Routing / Orchestration — The classification step that determines which retrieval source(s) — customer data, live docs, or both — a given question requires, before invoking the corresponding tools.

Contradiction Detection — A cross-source consistency check that flags cases where the customer-data corpus and live release notes disagree (e.g. a feature marked "requested" that is already shipped).

Fallback Model Routing — Automatically retrying a failed or unreliable primary-model request against a secondary model with an identical tool-calling contract, to preserve reliability without provider-specific branching logic.

6 Key Design Decisions

6.1 No Agentic Harness

Sanchalak's control flow (classify → retrieve → synthesize) is fixed and bounded, decided by orchestrator code rather than an open-ended model-driven loop. A general agent framework (e.g. an autonomous multi-step agent executor) was deliberately not used, since it would add operational complexity without addressing a genuine need for unbounded iteration.

6.2 Live Documentation Fetching, Not Static Scraping

Per the problem statement's explicit requirement, FlytBase's documentation and release notes are fetched live at query time rather than pre-scraped into a static index. This ensures answers

reflect the current published state of FlytBase's product documentation at all times.

6.3 Deterministic Tier for Simple Queries

Not every question requires model inference. Simple, filterable lookups are answered deterministically wherever possible, improving reliability, latency, and demo robustness by reducing dependence on external LLM APIs for the most common query pattern.

6.4 Strict Scope on Customer Data

In line with hackathon constraints, no real FlytBase customer data is used at any point — the system operates exclusively on the provided synthetic corpus, and no live connection to FlytBase's internal production systems is implemented.

7 Naming

The name **Sanchalak** is a Sanskrit/Hindi term meaning *operator*, *conductor*, or *the one who runs or drives something* — also used colloquially for a moderator or anchor. The name was chosen to reflect the system's core function: conducting a question to the right source(s), orchestrating retrieval across two otherwise disconnected systems, and driving a single grounded answer out of them.

8 Summary

Sanchalak demonstrates that a cross-source knowledge agent can be built as a disciplined, bounded pipeline rather than an open-ended autonomous system — combining a locally indexed synthetic customer corpus with FlytBase's live public documentation, enforcing strict grounding and citation discipline, supporting incremental updates without a full rebuild, and surfacing genuine cross-source contradictions as a first-class feature rather than an afterthought.